

Chapitre 8 – Tests unitaires

Sommaire

I.	Le principe	1
II.	Mise en œuvre avec Visual Studio.....	2
1.	Créer un projet.....	2
2.	Créer un projet de test.....	2
3.	Créer une première méthode de test	2
4.	Exécuter le(s) test(s).....	3
5.	Créer une méthode de test pour des exceptions	4
6.	Méthodes d'initialisation et de nettoyage - Principe.....	6
7.	Méthodes d'initialisation et de nettoyage - Démonstration	6
III.	Application.....	8

I. Le principe

Les tests unitaires ont pour objet de vérifier que chaque méthode de chaque classe à tester réagit comme elle le doit.

Pour cela, chaque méthode sera appelée individuellement, en respectant 3 phases :

- *Arrange* (préparer) ;
- *Act* (agir) ;
- *Assert* (vérifier) ;

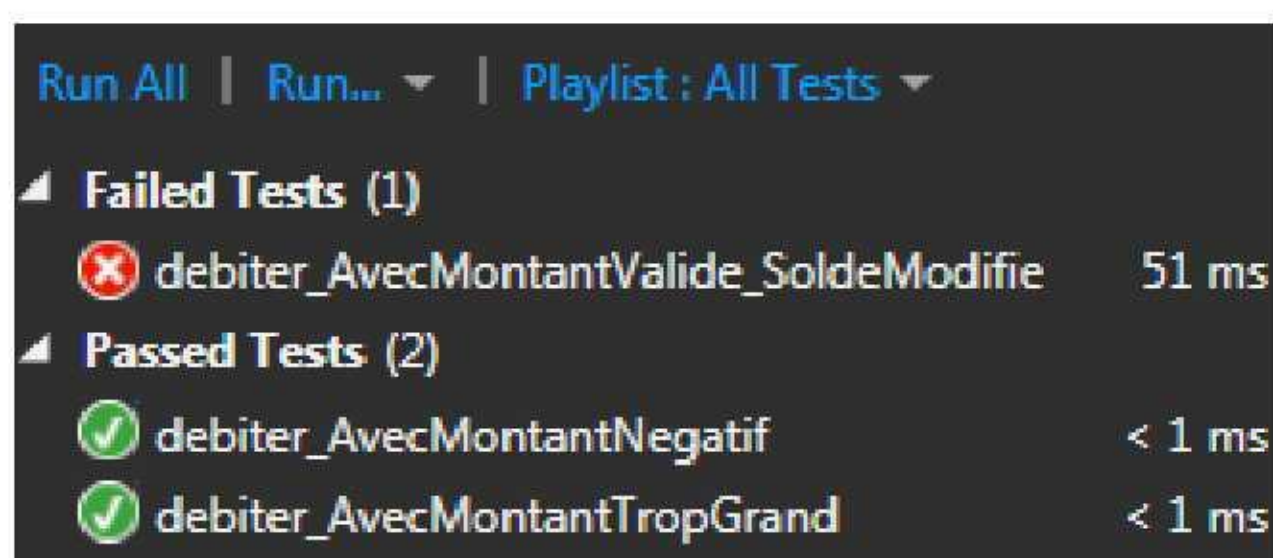
La **préparation** est variable suivant les cas, et consiste à créer les éléments (objets, variables) pour créer une situation de départ.

Agir, c'est appeler la méthode avec les éléments préparés avant.

Vérifier (assert), c'est comparer le résultat attendu avec le résultat renvoyé par l'appel précédent, et proposer un message affiché en cas de différence. Les succès ou erreurs de cette étape sont comptabilisés et visualisés.

L'objectif est donc de réaliser des tests unitaires qui **couvrent toutes les situations** pour chacune des méthodes testées (pour les classes testées), et qu'aucune erreur ne soit provoquée.

Exemple de résultat produit par les tests unitaires :



Certains, dans de rares cas, proposent de commencer à développer en écrivant les tests unitaires d'abord, puis en codant les classes testées ensuite. Cela permet d'analyser tous les cas qui peuvent se produire, et la réponse qui devrait y être apportée.

II. Mise en œuvre avec Visual Studio

Visual Studio utilise un framework de test nommé MSTest. On en trouve d'autres équivalents pour d'autres plateformes (JUnit notamment en Java).

1. Créer un projet

On crée un projet *Banque* de type console, reposant sur une classe unique : *CompteBanque*. Cette classe contiendra trois champs privés, un constructeur, deux propriétés en lecture seule, et trois méthodes.

- Créer un nouveau projet de type *Application Console*, nommé *Banque* ;
- Coller dans la méthode *main()* le source proposé dans *Méthode main.cs* ;
- Ajouter une nouvelle classe (*Add, New Item, Class* depuis le projet) nommée *CompteBanque.cs* ;
- Coller dans la classe le contenu du fichier *Classe CompteBanque.cs* ;
- Exécuter : La ligne **Solde du compte de Mme Julie ROUBON : 28,98 Euros** doit apparaître ;

2. Créer un projet de test

Dans la même solution, un projet de test est créé.

- Ajouter un projet de test à la solution (*Add, New Project, Test, Unit Test Project* depuis la solution) ;
- Nommer le projet *BanqueTest* ;
- Un source pour les tests apparaît :

```
using System;
using Microsoft.VisualStudio.TestTools.UnitTesting;

namespace BanqueTest
{
    [TestClass]
    public class UnitTest1
    {
        [TestMethod]
        public void TestMethod1()
        {
        }
    }
}
```

- Observer que la description de la classe est décorée de l'attribut *[TestClass]* ;
- Et que la méthode possède l'attribut *[TestMethod]* ;

Cette classe de test va servir à tester la classe *CompteBanque*. La norme veut qu'on ajoute *Test* pour donner le nom de cette classe :

- Changer le nom de la classe de *UnitTest1* en *CompteBanqueTest* ;
- Renommer le fichier pour qu'il corresponde à la classe : *CompteBanqueTest.cs* ;

3. Créer une première méthode de test

Les méthodes de tests ont presque toutes la même signature : Elles ne renvoient rien, et prennent rarement des paramètres.

- Observer la signature de *TestMethod1* ;
- Remplacer cette méthode par celle proposée dans *Méthode de test.cs* ;
- Observer les 3 parties :

- Arrange / Préparer : Initialisation de variables et création d'un objet ;
- Act / Agir : Appeler la méthode débiter dans une situation favorable (solde suffisant) ;
- Assert / Vérifier : Lire le solde actuel, et le comparer avec la valeur attendue (le 3^{ème} paramètre est l'écart toléré). En cas d'erreur, le message en 4^{ème} paramètre sera affiché ;

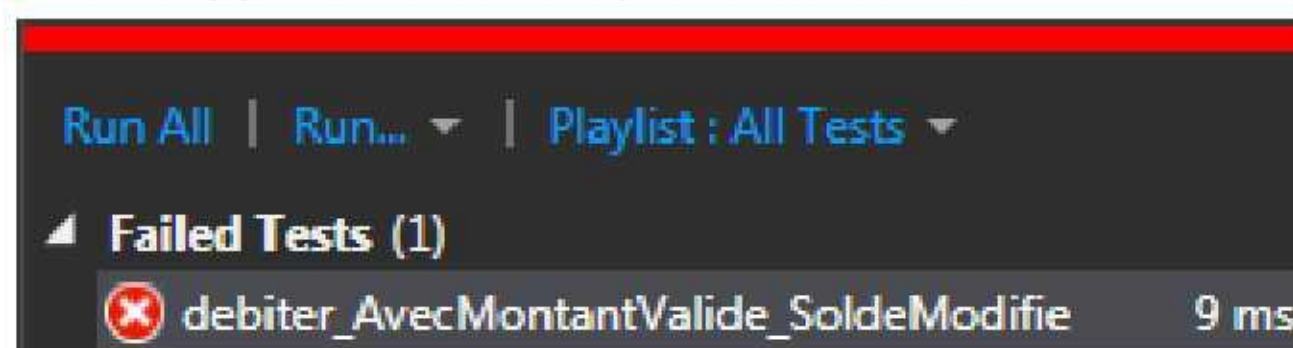
Pour que cela fonctionne, il faut que la classe *CompteBanque* du projet *Banque* soit accessible dans le projet *BanqueTest* :

- Depuis le projet *BanqueTest*, cliquer droit sur *References, Add References, Projects* ;
- Cocher *Banque*. Valider ;
- Ajouter en début la clause `using Banque;`
- La classe *CompteBanque* est accessible ;

4. Exécuter le(s) test(s)

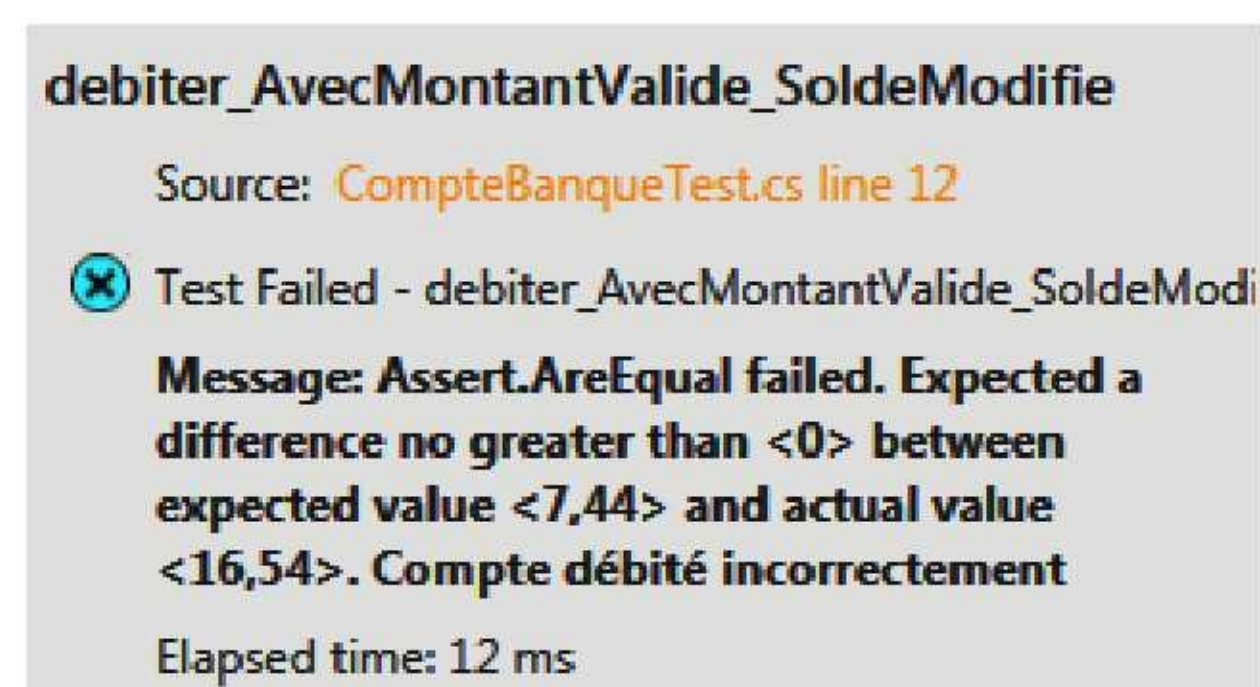
Les fonctions de test peuvent être toutes exécutées (conseillé), ou bien une partie seulement. Elles peuvent être regroupées en catégories pour les manipuler plus facilement, ...

- Choisir le menu *Test, Run, All Tests* ;
- La fenêtre *Test Explorer* apparaît (accessible par le menu *Test, Window*) ;
- Le test apparaît comme ayant échoué ! :



- 3 possibilités :
 - La méthode testée est mal écrite ;
 - La méthode de test est mal écrite ;
 - Les deux méthodes sont mal écrites ;

Pour y voir plus clair, observer le message en sélectionnant la méthode de test :



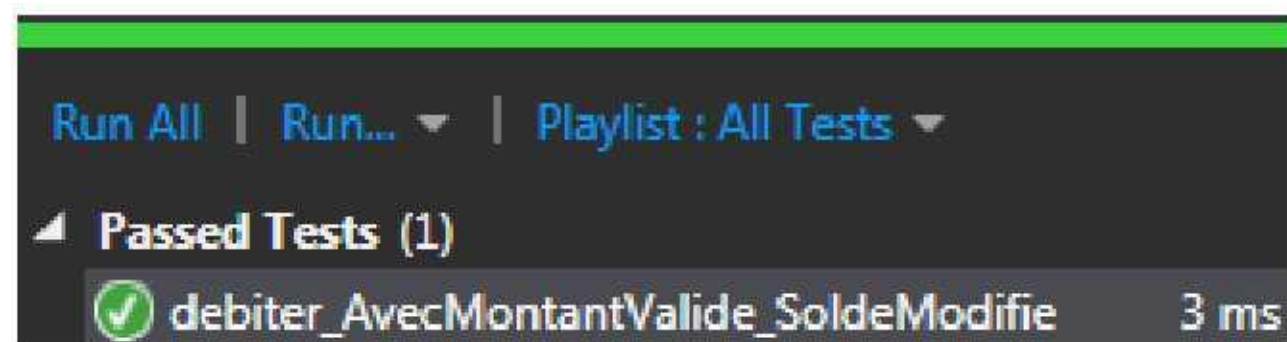
On s'attend à avoir la valeur 7.44 dans le solde, il y a en fait 16.54 !

Retrouver l'erreur et la corriger dans le source.

Solution :

```
public void debiter(double montant)
{
    //...
    m_Solde -= montant;
}
```


Relancer le test (cliquer sur *Run All* dans la fenêtre de test) :



5. Créer une méthode de test pour des exceptions

La méthode *debiter* lève dans certains cas des exceptions. Il faut vérifier que celles-ci ont bien lieu :

- Créer une deuxième méthode de test nommée *debiter_AvecMontantTropGrand* :

```
[TestMethod]
public void debiter_AvecMontantTropGrand()
{
    // Arrange - Préparer
    double soldeInitial = 11.99;
    double montantDebit = 14.55;
    double soldeAttendu = 11.99;
    CompteBanque compte = new CompteBanque("Mme Julie ROUBON", soldeInitial);

    // Act - Agir
    try
    {
        compte.debiter(montantDebit);
    }

    // Assert - Vérifier
    catch (ArgumentOutOfRangeException ex)
    {
        double solde = compte.solde;
        Assert.AreEqual(soldeAttendu, solde, 0, "Compte débité incorrectement");
        return;
    }
    Assert.Fail("Compte débité à tort");
}
```

- Bien l'analyser ;
- Pourquoi y a-t-il un *return* ?
- Vérifier son exécution ;

Proposer une/d'autre(s) méthode(s) de test à ajouter pour la méthode *debiter*.

Solution 1/2 : *debiter_AvecMontantNegatif* :


```

[TestMethod]
public void debiter_AvecMontantNegatif()
{
    // Arrange - Préparer
    double soldeInitial = 11.99;
    double montantDebit = -4.55;
    double soldeAttendu = 11.99;
    CompteBanque compte = new CompteBanque("Mme Julie ROUBON", soldeInitial);

    // Act - Agir
    try
    {
        compte.debiter(montantDebit);
    }

    // Assert - Vérifier
    catch (ArgumentOutOfRangeException ex)
    {
        double solde = compte.solde;
        Assert.AreEqual(soldeAttendu, solde, 0, "Compte débité incorrectement");
        return;
    }
    Assert.Fail("Compte débité à tort");
}

```

Solution 2/2 : *debiter_CompteCloture* :

```

[TestMethod]
public void debiter_CompteCloture()
{
    // Arrange - Préparer
    double soldeInitial = 11.99;
    double montantDebit = 4.55;
    double soldeAttendu = 11.99;
    CompteBanque compte = new CompteBanque("Mme Julie ROUBON", soldeInitial);
    compte.cloturer();

    // Act - Agir
    try
    {
        compte.debiter(montantDebit);
    }

    // Assert - Vérifier
    catch (Exception ex)
    {
        double solde = compte.solde;
        Assert.AreEqual(soldeAttendu, solde, 0, "Compte débité incorrectement");
        return;
    }
    Assert.Fail("Compte débité à tort");
}

```

La méthode *debiter_CompteCloture* doit mettre en évidence une erreur dans la méthode *debiter* corrigée ainsi :

```

public void debiter(double montant)
{
    if (m_Clature)
    {
        throw new Exception("Compte clôturé");
    }
    if (montant > m_Solde)

```


6. Méthodes d'initialisation et de nettoyage - Principe

Parfois, chaque méthode de test d'une classe nécessite une partie commune d'initialisation et une autre commune chargée du nettoyage.

Trois niveaux sont considérés : Au niveau méthode (*Test*), au niveau classe (*Class*) et au niveau module (*Assembly*). On a donc 6 possibilités pour ces attributs.

Au niveau de chaque méthode de test :

- `[TestInitialize]` : Une méthode avec cet attribut est appelée **avant chaque méthode** de test de la classe ;
- `[TestCleanup]` : Une méthode avec cet attribut est appelée **après chaque méthode** de test de la classe ;

Au niveau de chaque classe de test :

- `[ClassInitialize]` : Cet attribut provoque **un seul appel** de la méthode **avant toute méthode** de test de la classe ;
- `[ClassCleanup]` : Cet attribut provoque **un seul appel** de la méthode **après toute méthode** de test de la classe ;

Au niveau d'un module contenant des classes de test :

- `[AssemblyInitialize]` : Une méthode avec cet attribut est appelée **une seule fois en début** d'exécution ;
- `[AssemblyCleanup]` : Une méthode avec cet attribut est appelée **une seule fois en fin** d'exécution ;

Attention, la signature de ces méthodes change :

- `[ClassInitialize]` : La méthode doit être statique et doit avoir un paramètre de type *TestContext* ;
- `[ClassCleanup]` : La méthode doit être statique ;

7. Méthodes d'initialisation et de nettoyage - Démonstration

Dans les 4 méthodes de test précédentes a été ajouté un message vers la console :

```
[TestMethod]
public void debiter_AvecMontantValide_SoldeModifie()
{
    Console.WriteLine("debiter_AvecMontantValide_SoldeModifie");
    // Arrange - Préparer
    double soldeInitial = 11.99;
```

```
[TestMethod]
public void debiter_AvecMontantTropGrand()
{
    Console.WriteLine("debiter_AvecMontantTropGrand");
    // Arrange - Préparer
    double soldeInitial = 11.99;
```

```
[TestMethod]
public void debiter_AvecMontantNegatif()
{
    Console.WriteLine("debiter_AvecMontantNegatif");
    // Arrange - Préparer
    double soldeInitial = 11.99;
```

```
[TestMethod]
public void debiter_CompteCloture()
{
    Console.WriteLine("debiter_CompteCloture");
    // Arrange - Préparer
    double soldeInitial = 11.99;
```


Puis ont été ajoutées les 4 méthodes d'initialisation et nettoyage suivantes :

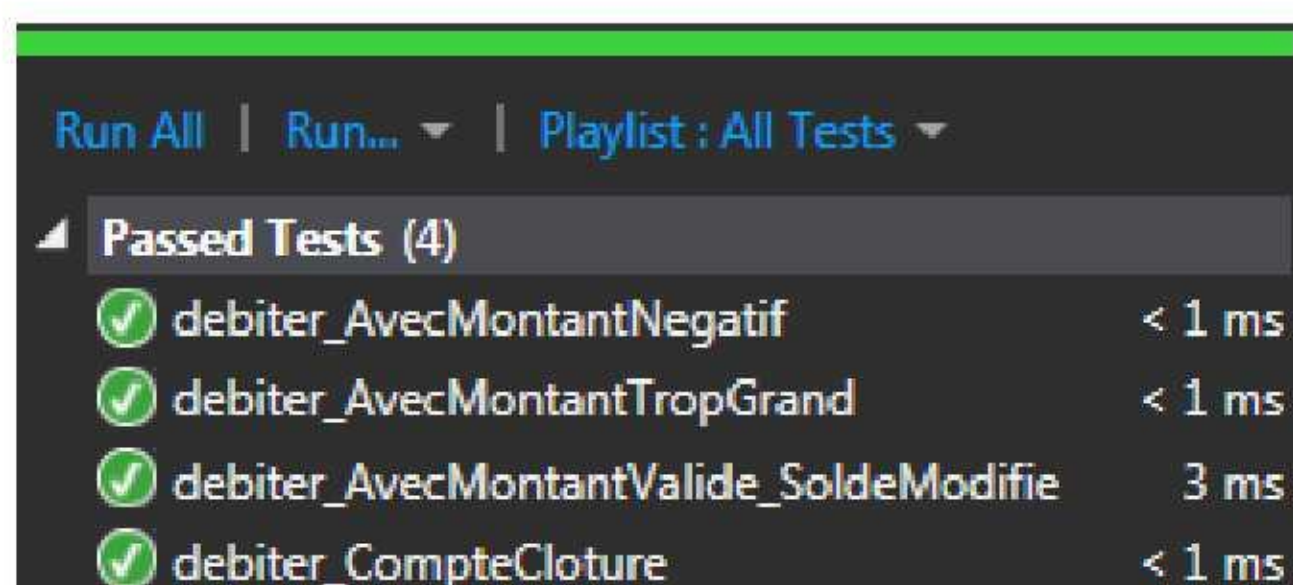
```
[TestInitialize]
public void debuterTest()
{
    Console.WriteLine("debuterTest");
}

[TestCleanup]
public void terminerTest()
{
    Console.WriteLine("terminerTest");
}

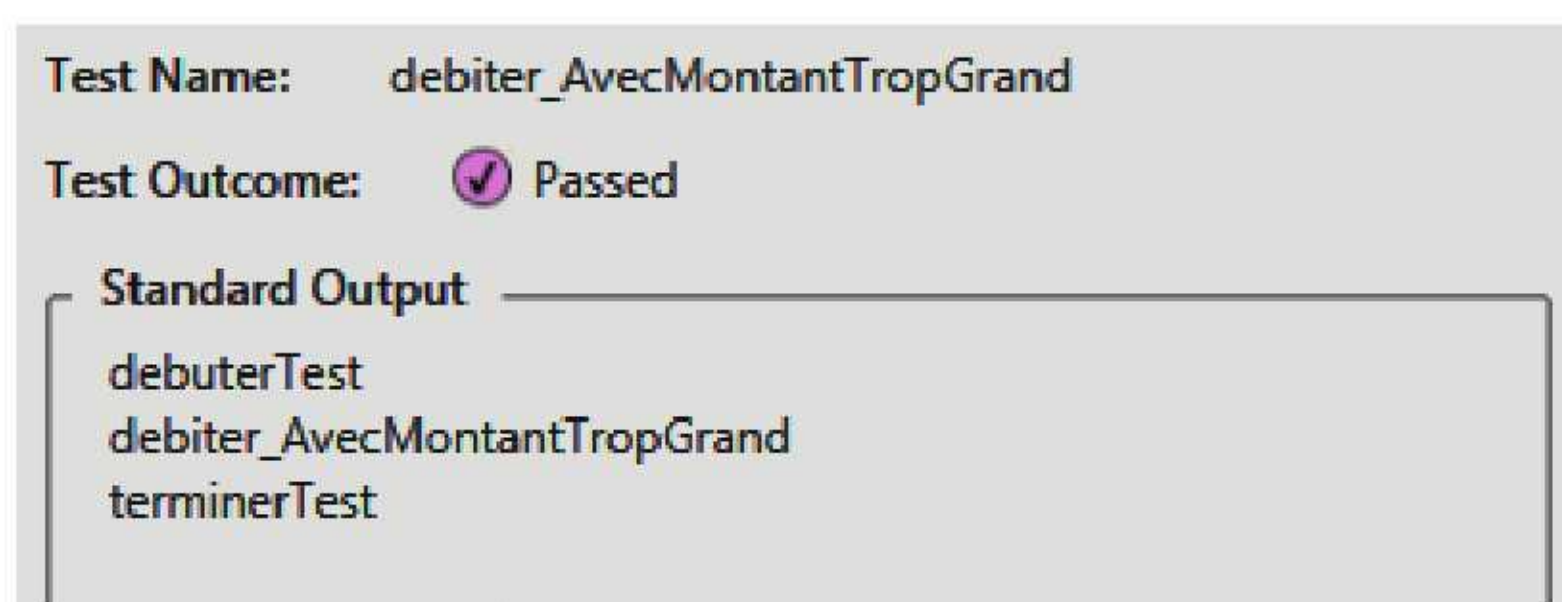
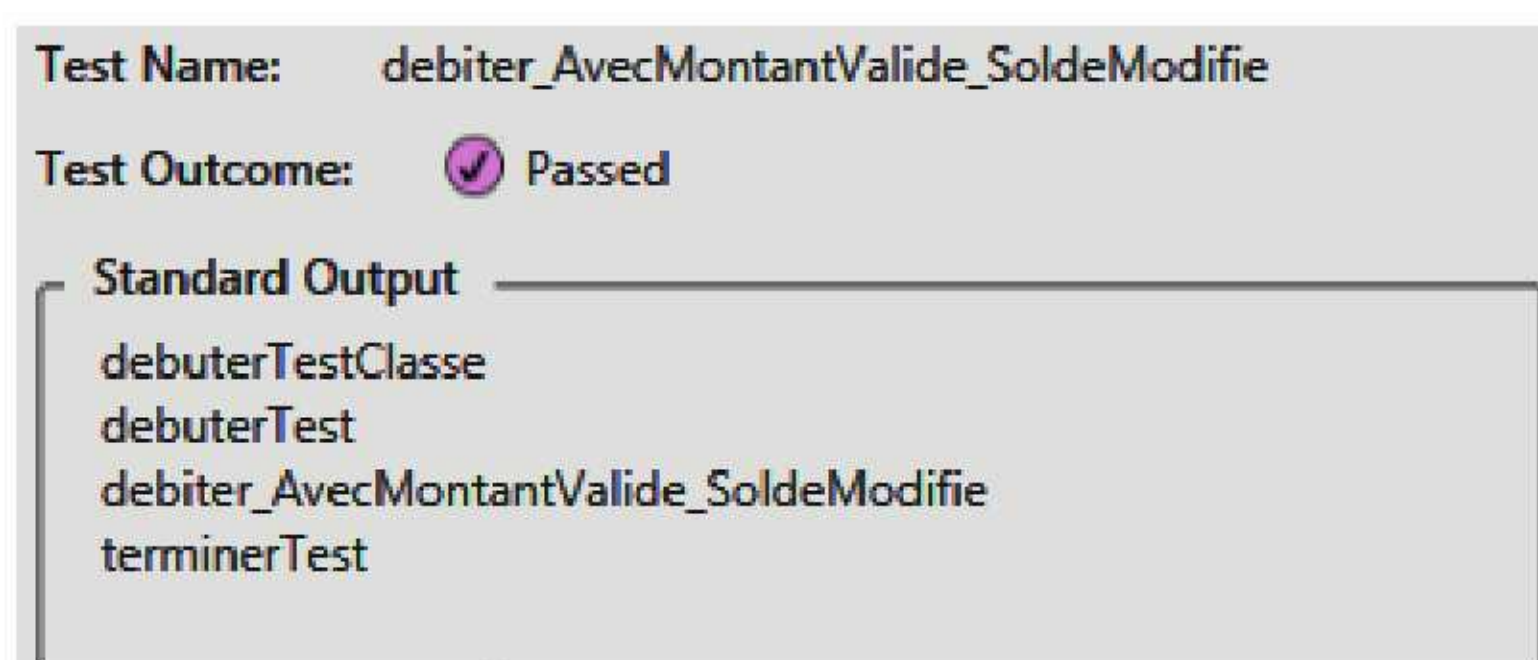
[ClassInitialize]
public static void debuterTestClasse(TestContext tc)
{
    Console.WriteLine("debuterTestClasse");
}

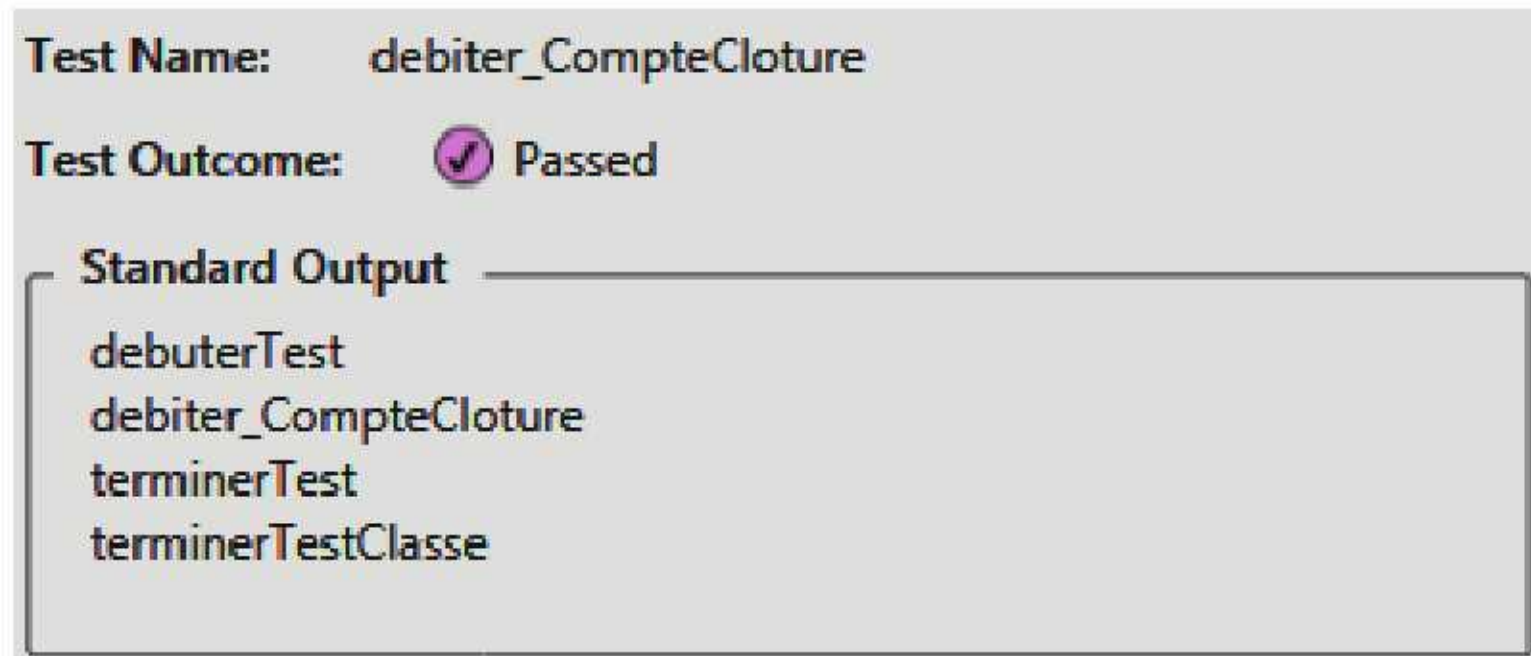
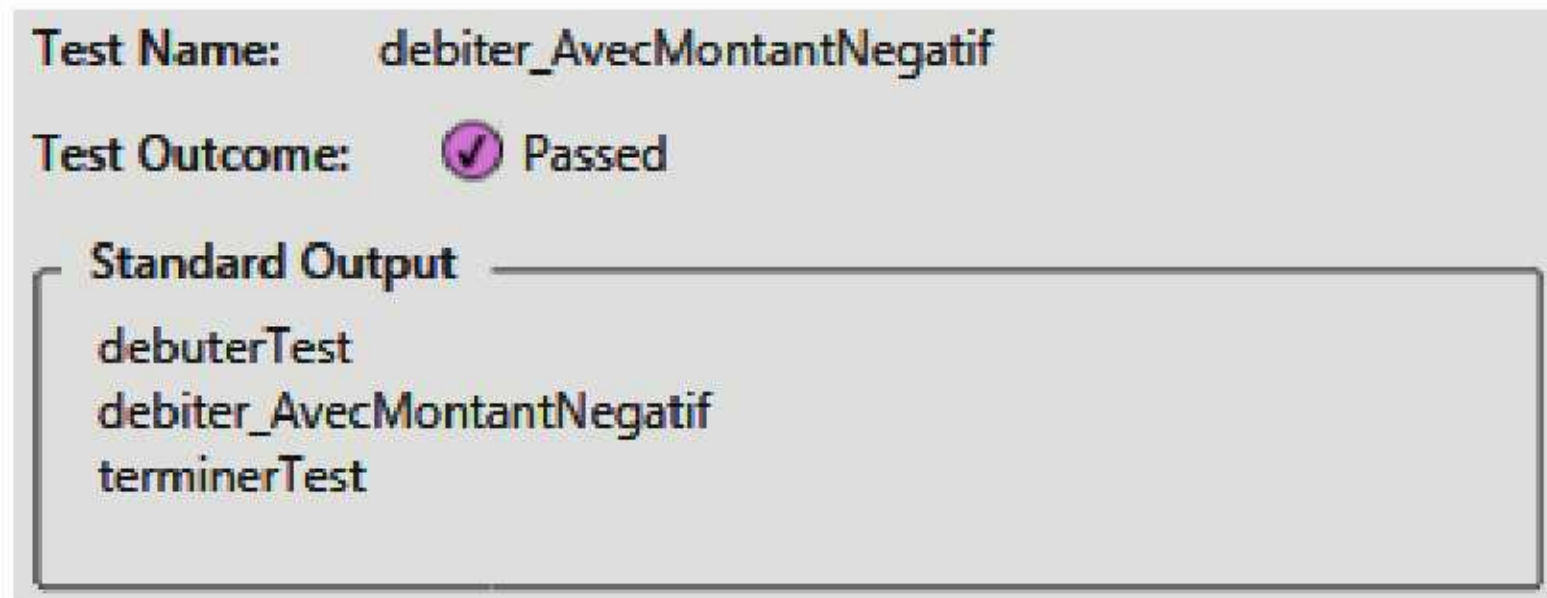
[ClassCleanup]
public static void terminerTestClasse()
{
    Console.WriteLine("terminerTestClasse");
}
```

A l'exécution des tests, 4 méthodes apparaissent :



La sortie (*output*) de chacune des méthodes donne :



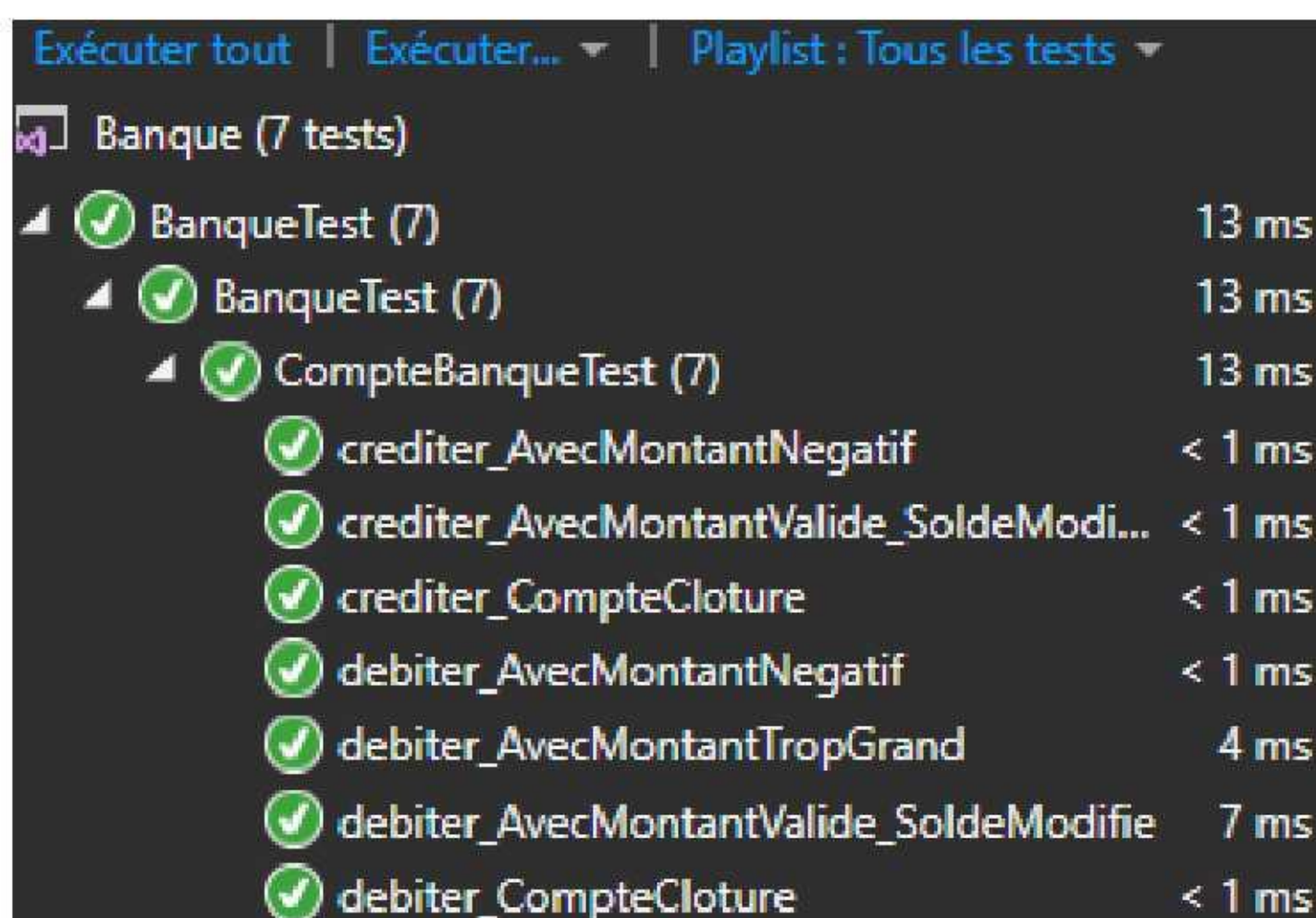


L'imbrication des appels est mise en évidence.

III. Application

Poursuivre les tests unitaires pour que la classe *CompteBanque* soit entièrement validée.

Résultats :



La méthode *crediter* est corrigée :

```
public void crediter(double montant)
{
    if (m_Clature)
    {
        throw new Exception("Compte clôturé");
    }
    if (montant < 0)
    {
        throw new ArgumentOutOfRangeException("montant");
    }
    m_Solde += montant;
}
```


Objectif des tests unitaires : Couvrir 100% du code des méthodes testées. *Visual Studio* en version payante propose un outil de couverture du code :

TEST ARCHITECTURE ANALYSER FEN...

Exécuter
Déboguer
Paramètres de test
Analyser la couverture du code
Windows

Tests sélectionnés
Tous les tests

Explorateur de tests

Exécuter | tout...

Tests réussis (3)

- QuickNonZero 15 ms
- RootTestNeg... 13 ms
- SignatureTest 1 ms

Non couvertes
Couvertes

```
public double SquareRoot(double x)
{
    if (x < 0.0)
    {
        throw new ArgumentOutOfRangeException();
    }
    double estimate = x;
    double previousEstimate = -x;
    while (System.Math.Abs(estimate - previousEstimate) >...)
    {

```

Activer la coloration

100 %

Résultats de la couverture du code

ctsoasm_MAIN50531 2012-06-07 02...

Hiérarchie	Non couv...	Non couvertes (%...	Couv...
ctsoasm_MAIN50531 201...	44	80.00%	11
fabrikam.math.dll	7	50.00%	7
{ } Fabrikam.Math	7	50.00%	7